

How to Do Evals in 2026

A tool-agnostic framework for evaluating AI agents



ALEXEY GRIGOREV

AUG 14, 2026



25



2

Share

Across all 4,894 AI engineering job descriptions that I collected for the [AI Engineering Field Guide](#), evaluation consistently comes as the number one skill that AI Engineers must have.

It also appears in interview questions. In interviews, you'll definitely hear something like "how do you evaluate a RAG pipeline?" or "how do you measure agent performance?". If you don't include evals in your home assignment, it's a red flag.

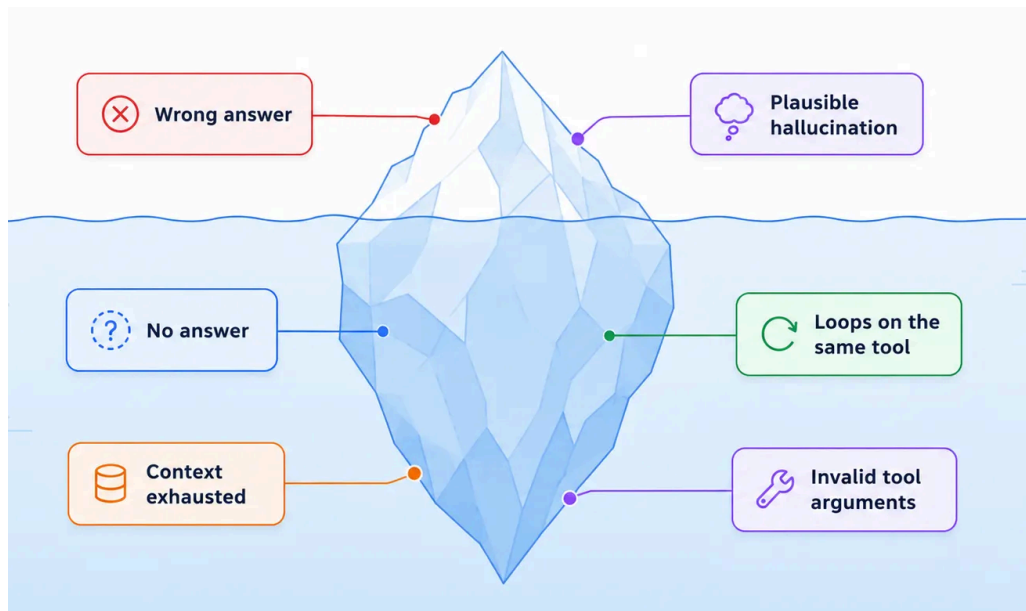
Why is it such an important skill? Because it's hard, and it matters.

It's easy to build an agent these days. I show how to do it in my ["AI Hero: 7-Day AI Agents Crash-Course"](#). You need to get an OpenAI key, take any agentic framework, define the instructions and the tools, and that's it. You have an agent.

But this agent will break in so many ways. It will:

- Give a wrong answer
- Confidently hallucinate a plausible answer
- Finish without giving any answer
- Get into a loop and call the same tool over and over again
- Exhaust the context
- Call a tool with invalid arguments

That's just the tip of the iceberg. To make sure this agent works reliably, you need to test and evaluate it.



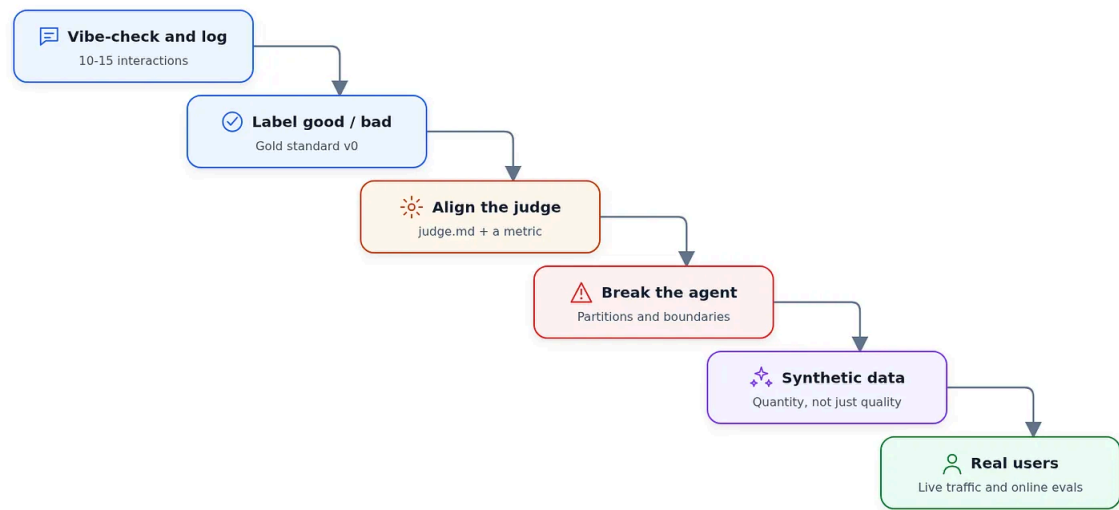
Also, without evaluations, you're blind. You don't know how the agent is performing. You also can't really change anything: every change you introduce may break the system in many ways, including the areas where you least expect it.

Agent evaluation is not simple. It takes a lot of time to understand the problem, get input from real users, and gather the right data. It's not something you can just delegate to Claude and forget about it.

That's what makes evals such an important part of AI Engineering.

In this article, I will tell you about my approach to evaluations:

- Start manually by "vibe-checking" the system, but also collect logs as early as possible
- Create a tool for labelling the logs and put together a gold standard dataset
- Create a judge that's aligned with our judgement
- Break the agent like a QA engineer
- Get more data by generating it synthetically
- Start collecting data from real users
- Monitor your system with online evaluation
- Always refine your gold standard dataset



The stages, in the order I go through them. Each stage adds new cases to the gold standard dataset.

Let's get into details.

Thanks for reading Alexey On Data! Subscribe for free to receive new posts and support my work.

Collecting the Gold Standard Dataset

You have an agent. What's next?

There's nothing wrong with "vibe-checking" it: you poke it, ask questions you want it to answer, and look at the results. If they aren't good, you figure out how to fix it.

But already at this step you can be a bit more organized and start logging what the agent is doing. When you ask a question and see the answer, make sure they are saved somewhere. You do a few vibe-checking sessions, and you already have 10-15 logged records.

Now you can systematically look at these logs and classify each record into

- "good": the answer is what you expect
- "bad": the answer is not what you wanted to see

To help me do it, I usually vibe-code a small labelling tool.

FAQ Assistant Eval Tool

Manual labeling for gold-standard dataset

Record 7 of 15

course: de-zoomcamp

source: Slack thread

record_id: slack-0007

User Question

How do I submit the final project for Data Engineering Zoomcamp?

Assistant Answer

Please check the course repository for the latest instructions.

⚠ Answer may be incomplete

Retrieved FAQ Entries

Final project submission

Submit your project through the course portal before the deadline. Upload the GitHub repository link and a short README with setup instructions and screenshots.

Project review process

After submission, peer review opens the following week. Make sure your repository is public and includes the data pipeline instructions.

Label

Good

Bad

Skip

Current label: **Bad**

The labelling tool for the “version 0” of the gold standard dataset

This gives you a “version 0” of the gold standard dataset - the dataset you will use for evaluating your system (sometimes it’s also called the “ground truth” dataset).

Don’t delegate this step to coding assistants. It’s only a few examples, so you can go through them relatively quickly. But this way, you will get a lot of insights into how your agent is behaving, what’s working and what isn’t.

Judge alignment

Once you manually labelled 10-15 examples, it’s time to start automating it.

I use coding assistants for that:

- Ask the assistant to analyze the logs and classify the examples into good or bad.
- See if the results match yours. If they don’t, ask the agent to explain its decision - maybe you missed something, or the agent didn’t have enough context.
- Iterate until you both agree.
- At the end of the session, ask the agent to create a judge.md file that describes the classification rules.
- Read this file carefully to see that the rules are generic and not specific to the examples you have in the dataset.

```
</> Markdown

# FAQ Assistant Judge

Classify each answer as GOOD or BAD.

## GOOD
- Directly answers the user's question
- Uses information supported by the retrieved FAQ
- Includes important steps or constraints
- Does not invent unsupported details

## BAD
- Gives incorrect information
- Is too vague to be useful
- Omits an essential step
- Contradicts the retrieved FAQ

When uncertain, prefer BAD.
```

An example of a judge.md file with generic classification rules

We just created a judge - a system that evaluates our agent.

Now we need to test that using judge.md alone is sufficient:

- Start a new session
- Ask to start a subagent which reads judge.md and classifies the records in your dataset
- If something is off, ask the agent to change judge.md
- Iterate until you both align on what's good and bad

This process is called "alignment" - we align the judge with what we think is good or bad.

It's the same as training a binary classifier that predicts your decisions. But instead of weights, you have a judge.md file. (And it's also helpful to have a train-test split here too! But later, when you have more data.)

At this point, you also get a metric - the fraction of good examples. Now if you change something in your agent, you can re-run the judge and make sure this metric doesn't go down.

```
MINGW64/c/Users/alexey x + v
- Dataset: generation_eval_v0
- Judge: judge.md
- Records evaluated: 15
- Good: 12
- Bad: 3
- Score: 80.0%
- Main failures: incomplete answers, omitted submission steps,
  and one retrieval miss.

Examples of BAD cases:
- Final project submission answer omitted key steps.
- Office hours answer was too vague.
- Minsearch KeyError question returned no useful retrieval.

Report saved to reports/eval_2026-08-14.md

Final authorization required: should I now inspect the 3 BAD cases,
propose retrieval or prompt fixes, and prepare a revised candidate?

- Worked for 2m 18s
> Inspect the 3 BAD cases |

gpt-5.6-sol medium · Context 29% left · ~/git/faq-assistant · Context 71% used · weekly 12%
[faq-assistant:main*] 17:26 14-Aug-26
```

Codex running the evaluation: 12 good, 3 bad, 80% score

Breaking the agent

We have the initial version of our gold standard dataset. But it's not enough. Now we want to find cases that break our system.

QA engineers have been breaking software for many decades. For coming up with cases that should derail our agent, we can borrow some concepts from their frameworks.

1. Equivalence partitions

The first step is [equivalence partitioning](#). You divide the input space into groups where inputs within each group should produce similar behavior.

If we have a RAG agent that answers questions about our docs, it could be:

- Questions about topics in the documentation,
- Questions about relevant topics but not in the documentation,
- Questions about irrelevant topics,
- Ambiguous questions,
- Questions in different languages

Once you come up with a list of groups, come up with 2-3 questions from each group. You can ask AI to help you with the entire process.

2. Boundary testing

The next concept is [boundary testing](#).

We already have some questions from the previous step, but we can be even more aggressive in our testing.

After defining the partitions, QA engineers would test the boundaries of each partition. We can do the same.

For a RAG agent, come up with queries that:

- Exactly match a document that we have in our database (easy case),
- Are semantically related but use different words (typical case),
- Are semantically related to multiple documents (ambiguous case),
- Are completely off-topic (boundary case).

	A	B	C	D	E
1	id	course	question	kind	expected
2	1	de-zoomcamp	When is the homework 3 deadline?	in_docs	answer
3	2	de-zoomcamp	How do I submit the final project?	in_docs	answer
4	3	de-zoomcamp	Is there a certificate for this course?	not_in_docs	not_found
5	4	de-zoomcamp	How do I submit it?	ambiguous	clarify
6	5	de-zoomcamp	What's the weather in Berlin today?	off_topic	off_topic
7	6	de-zoomcamp	¿Cuál es la fecha límite de la tarea 3?	different_language	answer
8	7	de-zoomcamp	When is the homework 3 deadline?	exact_match	answer
9	8	de-zoomcamp	What's the due date for HW3?	paraphrase	answer
10	9	de-zoomcamp	hw3 deadline?	short_query	answer
11	10	de-zoomcamp	How do I set up the project?	multiple_matches	answer
12	11	de-zoomcamp	How do I install Python?	near_off_topic	off_topic
13	12	de-zoomcamp	Can I submit homework after the deadline?	boundary	answer
14	13	de-zoomcamp	Where do I submit homework? I don't see it.	in_docs	answer
15	14	de-zoomcamp	I get KeyError: 'course' when running index.	technical	answer
16	15	de-zoomcamp	What if I missed the deadline because of the weekend?	boundary	answer

Test questions from equivalence partitions and boundary testing, tagged by kind and expected outcome

Further alignment

This way you can easily add 30-40 more questions to your gold standard dataset.

Let's take all the data we have, and run it against our agent. Then use the judge to classify the output.

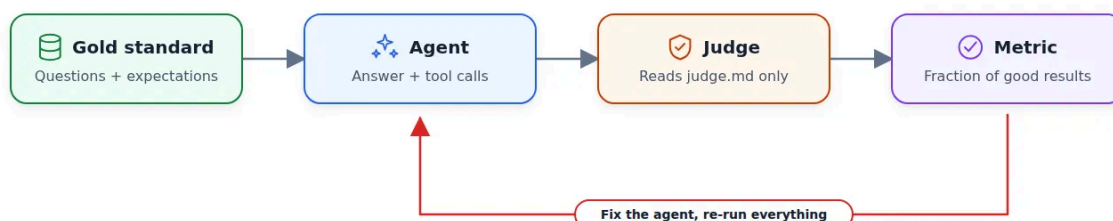
We already have a tool that we created earlier for manual evaluation. At this point, I adjust this tool to also support checking the output of the judge. For each record, I just decide whether I "agree" or "disagree" with the judge. I find it more convenient than labelling the new logs as "good" or "bad".

If we have many “disagree” cases, tune the judge following the same process as before. We will never have 100% alignment, so we don’t need to be too strict about it, but the judge should be correct most of the time.

Using the Judge to improve your agent

Now we have a fully automated process that:

1. Runs the agent against our data
2. Runs the judge against the output of step 1
3. Outputs a metric - the fraction of “good” results



The automated evaluation pipeline: fix and re-run

Because it’s automated, we can use it to improve our agent. You can ask the coding assistant to do it.

I recommend following the [implementer-tester pattern](#) for that:

- Define the implementer subagent that modifies the agent
- Define the judge subagent that uses judge.md to evaluate the output
- Iterate until your “good” ratio is acceptable


```
MINGW64:/c/Users/alexey/git  X  MINGW64:/c/Users/alexey  X  +  v
Metric    Before    After    Delta
Good      32/40 (80.0%)  36/40 (90.0%)  +10.0 pts
Bad       8/40 (20.0%)  4/40 (10.0%)  -10.0 pts
Skipped   0/40 (0.0%)  0/40 (0.0%)  0
Avg time  3.3s       3.1s       -0.2s

No regressions detected.
The revised candidate passes the regression check.

Report saved to reports/eval_2026-08-14_iter2.md

Should I inspect the remaining 4 failures and propose another revision?

*Cogitated for 11s

> |

▶▶ bypass permissions on (shift+tab to cycle) · ◀ for agents
```

The implementer-tester loop: one session changes the agent, the other one evaluates it

Synthetic data

So far we focused on the quality of our gold standard dataset. In this section, I want to talk about the quantity.

Quantity is also important because agents are non-deterministic. Even if the agent works properly on our small dataset, it will break when we get more traffic - just because of probability and statistics. When you expose your agent to a bigger number of inputs, things will break: with more repetitions, you increase the chances of something going wrong. That's why you want quantity here.

We can generate a lot more data using AI. Open your AI assistant, ask it to analyze your code, describe to it your target users, and then tell it to generate questions that these users are likely to ask.

For a RAG system, you can:

- Take a document from your knowledge base
- Ask AI to come up with 5 questions based on this document
- Do it for all documents

You will end up with a lot of questions. You can keep all of them, or take a sample.

	A	B	C	D	E
1	id	course	faq_id	question	kind
2	1	de-zoomcamp	de_hw_deadlines	When is homework 3 due?	paraphrase
3	2	de-zoomcamp	de_hw_deadlines	What's the deadline for HW3?	paraphrase
4	3	de-zoomcamp	de_hw_deadlines	Can I submit homework 3 after the deadline?	boundary
5	4	de-zoomcamp	de_hw_deadlines	What timezone are the homework deadline?	detail
6	5	de-zoomcamp	de_project_submit	How do I submit the final project?	paraphrase
7	6	de-zoomcamp	de_project_submit	Where do I send my project link?	paraphrase
8	7	de-zoomcamp	de_project_submit	Do I need to submit a GitHub repo for the final project?	detail
9	8	de-zoomcamp	de_project_submit	Can the final project repository be private?	boundary
10	9	de-zoomcamp	de_project_review	How does peer review work for the project?	paraphrase
11	10	de-zoomcamp	de_project_review	When does project peer review start?	detail
12	11	de-zoomcamp	de_project_review	How many projects do I need to review?	detail
13	12	de-zoomcamp	de_office_hours	How do I join office hours?	paraphrase
14	13	de-zoomcamp	de_office_hours	Where is the Zoom link for office hours?	paraphrase
15	14	de-zoomcamp	de_office_hours	Are office hours at the same time every week?	boundary
16	15	de-zoomcamp	de_setup_python	Which Python version should I use for the course?	paraphrase
17	16	de-zoomcamp	de_setup_python	Does Data Engineering Zoomcamp work with Windows?	boundary
18	17	de-zoomcamp	de_setup_docker	Do I need Docker for the course?	paraphrase
19	18	de-zoomcamp	de_setup_docker	Can I do the course without Docker Desktop?	boundary
20	19	de-zoomcamp	de_setup_wsl	Should I use WSL on Windows?	paraphrase
21	20	de-zoomcamp	de_setup_wsl	I'm on Windows. Do I need WSL2?	paraphrase
22	21	de-zoomcamp	de_gcp	Do I need a Google Cloud account?	paraphrase

Generating synthetic questions from a document in the knowledge base

Then follow the same process: run your agent against this data and then use the judge to evaluate the results.

This way you will probably run into many different problems:

- The agent will loop without stopping and exhaust the context
- It won't follow the instructions and do something that it shouldn't
- A tool will break and your agent won't finish the task

And many others. You don't know in advance what you'll get, which is why you want to expose the agent to a wide variety of inputs.

Once you identify the scenarios where your agent breaks or doesn't perform well, add them to your gold standard dataset.

Then you can fix the agent, and re-run the whole evaluation set to make sure you don't introduce any regressions.

Start testing on real users as soon as possible

We have evaluated our agent on many cases, so we're confident that it's working okay. But we always need more data.

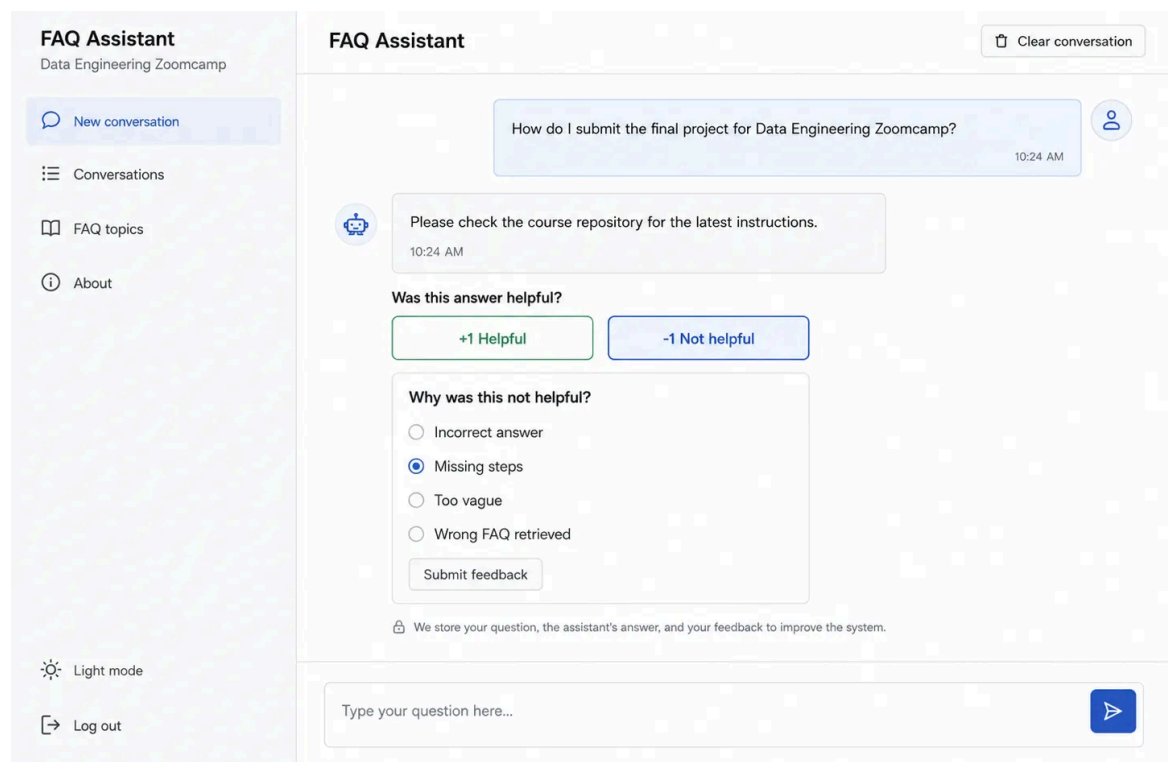
Now it's time to put it in front of real users:

- Invite people for user testing sessions to check your system

- If you plan to integrate it into an existing product, include it in a small subset of the traffic
- If it's a small personal project, ask your friends and family to test it.
- If it's a course project, ask your classmates to play with it.
- Share it on social media.

Make sure you're collecting the logs (be direct with the user about what you're collecting) and use this for adding new cases into the gold standard dataset.

Add +1/-1 buttons. This way, the users can give you explicit feedback, and if somebody dislikes the answer, you should inspect it.



The deployed agent in front of real users, with feedback buttons under every answer

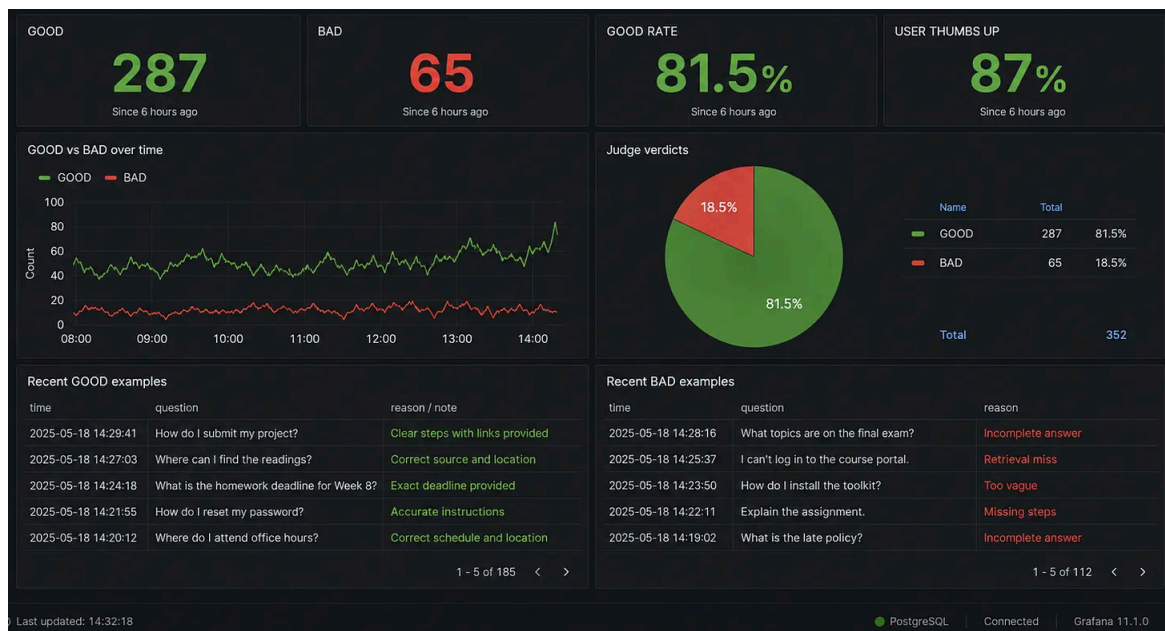
Online evaluation

We have a system for offline evaluations: we run the agent against our dataset and we use the judge to score the output to see how well it behaves.

It's okay, but it's not scalable. Instead, we want to run it continuously: every time somebody uses our agent, we react in real time - save the logs, evaluate them, and display the results on the dashboard.

We can't really continue using our coding agent for that, so we'll need to create a stand-alone judge.

It will give you a real-time view of the agent's quality: the performance on live traffic, and new cases that you can add to the gold standard dataset as they appear.



Online evaluation dashboard: judge evaluates the live traffic

Periodically inspect bad results, and if you see something new, add these cases to your gold standard dataset. Also sample the good results to make sure they are actually good.

For inspecting this live traffic I also often use coding assistants. I take a sample of the data, save it locally and ask them to go through these cases with me.

Multiple judges

We started with a good/bad evaluator, but I didn't really explain what it means. And it anyways depends on your problem.

Over time, we will need to have multiple judges, each evaluating the agent from a different angle:

- Task completion: the agent actually solved the user's problem
- Relevance/Correctness: the answer is relevant/correct
- Groundedness: the claims in the answer are supported by the documents in the knowledge base
- Completeness: the answer includes all the required information
- Following instructions: the agent follows the instructions correctly
- Trajectory optimality: the sequence of tool calls the agent used to complete the task is an optimal one



One agent run, several judges, one metric per dimension

There are other criteria too that you can consider for your application.

But I usually recommend starting with one judge and then adding others as your system matures.

The agent evaluation checklist

As a summary for this article, I wanted to give you a checklist that you can use next time you work on a new agent.

Gold standard dataset:

- [] Vibe-check the agent and log 10-15 interactions
- [] Label results as "good" or "bad" yourself
- [] Define equivalence partitions and test cases within each
- [] Include boundary cases (exact match, paraphrase, multiple matching documents, off-topic)
- [] Generate synthetic questions

The judge:

- [] judge.md describes the classification rules in generic terms, not specific to your examples
- [] The judge runs as a subagent that reads only judge.md
- [] Check the judge's answers with "agree" or "disagree"
- [] Use the fraction of "good" results as your metric
- [] Add more judges as your system matures

Improving the agent:

- [] The evaluation pipeline is automated: run the agent, run the judge, get the metric
- [] Each time you make a change, run the full evaluation set to catch regressions

Online evaluation:

- [] Put the agent in front of real users
- [] Collect user feedback
- [] Score online traffic with a stand-alone judge, show results on a dashboard
- [] Inspect bad results periodically, add new failing cases to the eval dataset

If you want to learn more about building AI Agents, I teach that in my course [AI Engineering Buildcamp: From RAG to Agents](#). In the course, I also cover evaluation in a lot of detail with real code, real tools, and real agents.

The next cohort starts September 21. You can use code "SUBSTACK" to get 20% off. See you there!

Subscribe to Alexey On Data

Launched 9 months ago

Practical writing on building, testing, and operating AI systems, workflows, and automation, based on my projects and experiments. Subscribe to get ideas you can reuse in your own work.

✓ **Subscribed**

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



25 Likes • 2 Restacks

← Previous

Next →

Discussion about this post

Comments Restacks



Write a comment...

